

Национальный исследовательский университет ИТМО
Мегафакультет компьютерных технологий и управления
Факультет программной инженерии и компьютерной техники

Отчет по практическому заданию №2
по курсу «Языки программирования»

Вариант: 3

Выполнил студент группы Р4119

(подпись)

В.Ю. Суровикин

Руководитель

(подпись)

Ю. Д. Кореньков

03 декабря
2025 г.

Санкт-Петербург
2025

Оглавление

1. Цель работы	3
2. План работы	3
3. Ход работы	4
3.1. Разработка синтаксического анализатора	4
3.2. Интерфейс разработанной программы	6
3.3. Примеры работы программы	6
4. Вывод	10
5. Исходные файлы	11

1. Цель работы

Реализовать построение графа потока управления (CFG) на основе результатов синтаксического анализа набора входных файлов, выполнить анализ полученной информации и сформировать набор файлов с графическим представлением графов потока управления для подпрограмм, а также графа вызовов между ними.

2. План работы

1. Описать структуры данных для представления набора входных файлов, подпрограмм и графов потока управления (включая базовые блоки и деревья операций).
2. На основе деревьев разбора, получаемых из модуля, разработанного в задании 1, реализовать модуль построения графа потока управления и регистрации логических ошибок.
3. Разработать тестовую программу, которая:
 - a. читает из аргументов командной строки имена входных файлов и выходную директорию,
 - b. вызывает модуль из задания 1 для получения деревьев разбора,
 - c. использует модуль построения графов потока управления для всех подпрограмм,
 - d. сохраняет графы потока управления и граф вызовов в выходные файлы.
4. Реализовать тестовую программу для демонстрации работоспособности созданного модуля.

3. Ход работы

3.1. Разработка синтаксического анализатора

В рамках выполнения данного задания будут использоваться структуры данных, приведенные ниже.

Структура данных, хранящая дерево разбора не претерпела изменений с прошлого задания:

```
struct ASTNode {
    std::string name;    // тип узла
    std::string value;   // значение
    std::vector<std::shared_ptr<ASTNode>> children;

    ASTNode(const std::string& n) : name(n) {}
    ASTNode(const std::string& n, const std::string& v) : name(n), value(v) {}

    void addChild(std::shared_ptr<ASTNode> child) {
        if (child) children.push_back(child);
    }
};

using ASTNodePtr = std::shared_ptr<ASTNode>;
```

Рисунок 1 - Структура узла дерева разбора

В этой лабораторной работе была разработана структура для хранения набора файлов, которые поступают на вход программы.

```
struct FileUnit {
    std::string fileName;
    ASTNodePtr root;
};
```

Рисунок 2 - Структура узла файла

Она хранит в себе имя переданного файла, а также дерево разбора для него. Далее разработана структура для хранения атрибутов подпрограммы.

```
struct FunctionInfo {
    FunctionSignature signature;
    std::string sourceFile;
    ControlFlowGraph cfg;
    ASTNodePtr funcAst;
};
```

Рисунок 3 - Структура узла информации подпрограммы

В ней хранится сигнатура подпрограммы (имя, передаваемые аргументы, возвращаемый тип данных), название файла, в котором она существует, а также граф потока управления, о котором будет сказано далее, и поддерево разбора для этой подпрограммы. Также разработана структура, которая хранит в себе граф потока управления .

```
struct ControlFlowGraph {
    std::vector<BasicBlockPtr> blocks;
    BasicBlockPtr entry;
    BasicBlockPtr exit;
};
```

Рисунок 4 - Структура узла графа потока управления

Сам граф потока управления и состоит из блоков, в одном узле хранится информация о текущем блоке, указатель на прошлый блок и следующий. Блок хранит в себе идентификатор, метку, дерево операций и коллекцию переходов.

```
struct BasicBlock {
    int id = -1; // уникальный ID внутри функции
    std::string label; // метка: "entry", "while.body1" и т.п.
    std::vector<OperationNodePtr> operations; // дерево(a) операций этого блока
    std::vector<CFGEdge> edges; // исходящие переходы
    bool isTerminal = false; // из блока дальнейший контроль не идёт (break, бесконечный цикл и т.д.)
};
```

Рисунок 5 - Структура узла базового блока

3.2. Интерфейс разработанной программы

Для работы с программой был обновлен интерфейс, представленный в лабораторной работе 1, была добавлена возможность указать несколько входных файлов, а выходной теперь указывается не файл, а директория, куда помещаются все файлы, полученные в ходе проекта.

Программа имеет следующий формат запуска:

```
ProgLangLab1 <input_file1> [input_file2 ... input_fileN] [-o <output_directory>]
```

Рисунок 6 - Интерфейс тестовой программы

Тестовая программа требует на вход хотя бы один файл с текстом на целевом языке. Можно подать на вход любое количество файлов. Также можно указать директорию, куда будет складываться результат. Если этого не сделать, результат сохранится там же, где лежат исходные тексты программ, поданных в качестве аргументов строки. Пример ввода команды:

```
ProgLangLab1 src/main.lang src/math.lang src/io.lang
```

Рисунок 7 - Пример использования

Программы будут разобраны, а файлы с описанием графического представления расположатся рядом с текстом программы. Для того чтобы получить именно графическое представление, был разработан скрипт, который преобразует графическое описание dot формата в png формат.

3.3. Примеры работы программы

Для проверки работоспособности программы был написан комплексный пример, охватывающий все конструкции языка:

Листинг 3.1. Программа на целевом языке

```
function isPositive(x as int) as bool
    dim temp as int
    if x > 0 then
        temp = x;
    else
        temp = 0;
    end if
end function

function sumPositive(n as int) as int
    dim i as int
    dim sum as int

    i = 0;
    sum = 0;

    do
        if i > n then
            break;
        end if

        if i > 0 then
            sum = sum + i;
        end if

        i = i + 1;
    loop while i <= n
end function

function findFirstPositive(arr as int(), size as int) as int
    dim i as int
    dim index as int

    i = 0;
    index = 0;

    while i < size
        if isPositive(arr(i)) then
            index = i;
            break;
        else
            i = i + 1;
        end if
    wend
end function

function processThreshold(arr as int(), size as int, threshold as int)
    dim i as int
    dim count as int

    i = 0;
    count = 0;

    do
        if i >= size then
            break;
        end if

        if arr(i) > threshold then
            count = count + 1;
        end if

        i = i + 1;
    loop until i >= size
```

```

end function

function main()
  dim n as int
  dim result as int
  dim arr as int()
  dim i as int
  dim firstPosIndex as int

  n = 10;
  result = sumPositive(n);

  i = 0;
  while i < n
    if i < 5 then
      arr(i) = i;
    else
      arr(i) = -i;
    end if
    i = i + 1;
  wend

  firstPosIndex = findFirstPositive(arr, n);

  processThreshold(arr, n, 3);
end function

```

Рассмотрим подпрограммы по порядку их появления в примере. Первой выступила функция isPositive.

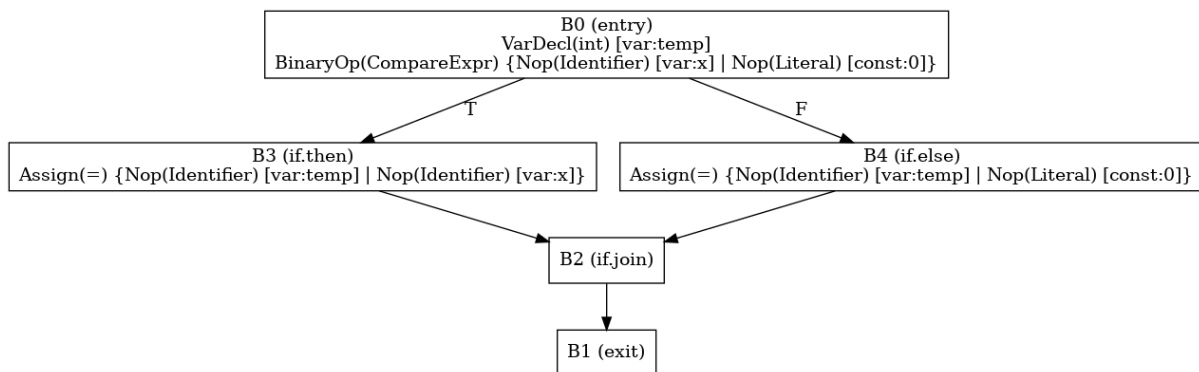


Рисунок 8 - Граф потока управления для isPositive

В этой функции можно увидеть как в общем работает модуль. Граф потока управления представлен в виде соединенных узлов, в качестве которых выступают базовые блоки. Они же хранят в себе информацию об условно или безусловном переходе, а также хранит в себе дерево операций, которое у меня представлено в плоском виде (последовательность фигурных скобок и вертикальных линий). Здесь можно увидеть, как раскладывается стандартный переход с условием (if/else).

Пойдем далее, к подпрограмме `sumPositive`. Здесь мы можем увидеть, как модуль раскладывает циклы вида `do-loop while`, а также как он обрабатывает `break`.

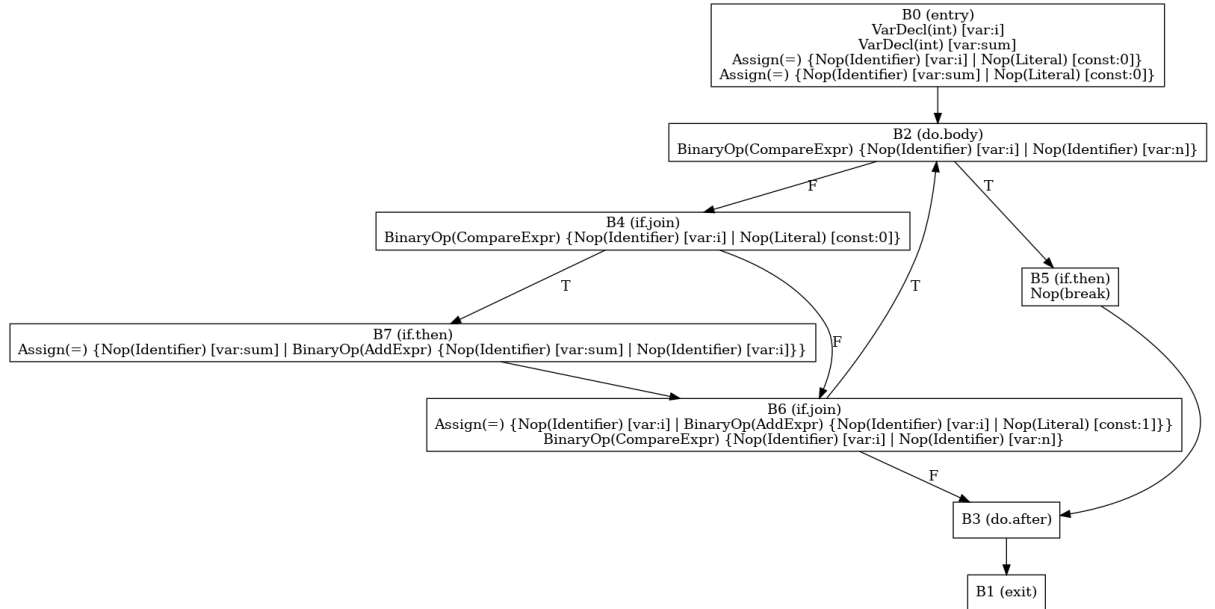


Рисунок 9 - Граф потока управления для `sumPositive`

Перейдем к подпрограмме `findFirstPositive`. Здесь мы можем увидеть, как модуль раскладывает цикл `while`, а также как он раскладывает вызов функции и обращение к элементу массива. В грамматике вызов и обращение к элементу описаны одинаково, и как мы можем увидеть здесь, они раскладываются похожим образом, но на самом деле при обработке этих конструкций в алгоритме, вызов имеет служебную информацию, позволяющую его отличать от обращения к элементу массива.

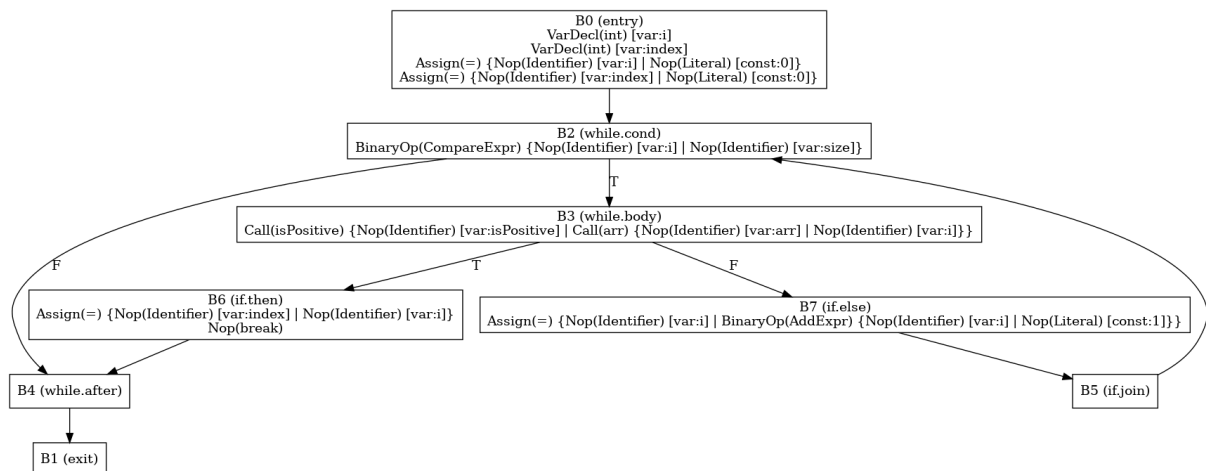


Рисунок 10 - Граф потока управления для `findFirstPositive`

Функцию processThreshold мы пропустим, так как она не имеет никаких новых конструкций, которых бы не было в прошлых примерах. Наконец-то мы рассмотрим, как все вызовы этих функций раскладываются в главной функции main.

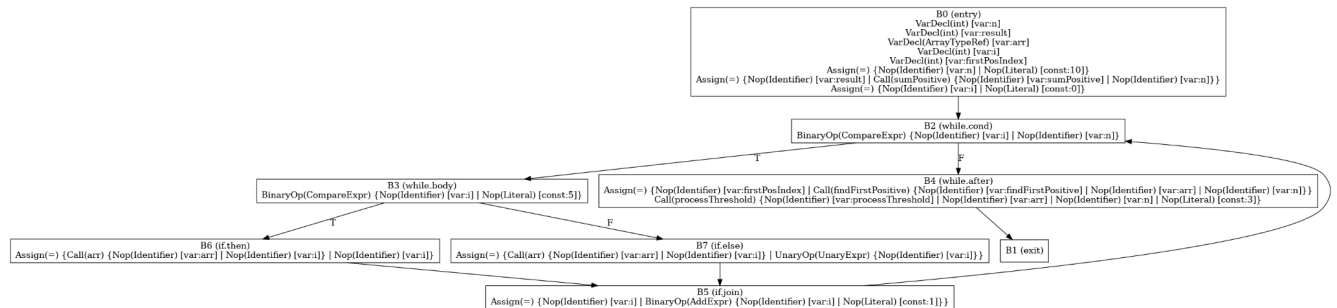


Рисунок 11 - Граф потока управления для main

Как мы видим, граф потока управления для main, в котором вызывается другие функции, выполняется корректно. Также мой модуль строит граф вызовов подпрограмм.

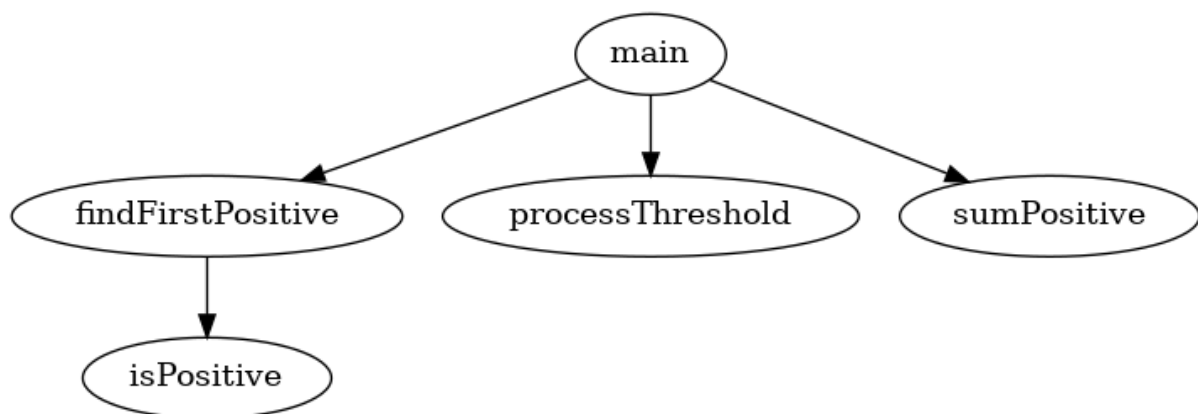


Рисунок 12 - Граф вызовов

Отсюда видно, в каком порядке вызываются функции. Также важно отметить, что перегрузка в данной реализации модуля не предусмотрена и вызывает ошибку, еще хочется указать на то, что объявление функции обязано быть раньше её реализации, иначе также будет ошибка.

4. Вывод

В ходе выполнения лабораторной работы был разработан и реализован модуль построения графа потока управления на основе синтаксических деревьев, получаемых из парсера, созданного в первой

лабораторной работе. Были спроектированы и реализованы структуры данных базовых блоков и всего графа потока управления, а также предусмотрены механизмы накопления информации об ошибках, вызовах подпрограмм и хранения информации о разных программах. На основе этих структур формируются CFG для каждой функции, а также сопутствующие данные, которые будут использованы в последующих этапах.

Была создана тестовая программа, позволяющая обрабатывать несколько входных файлов, а также сохранять результаты работы в виде графов потока управления и графом вызовов. Программа корректно обрабатывает широкий спектр конструкций языка: константы, выражения (включая индексные и скобочные), условные операторы с и без else, блоки begin–end, циклы while и do/loop, а также оператор break с дополнительной очисткой недостижимых блоков.

Проведённые эксперименты на тестовых примерах показали работоспособность разработанного модуля, корректность построения графов потока управления и графа вызовов, а также адекватную регистрацию и вывод ошибок. Таким образом, цель лабораторной работы достигнута: создан модуль построения CFG и графа вызовов, интегрированный с синтаксическим анализатором и обеспечивающий основу для дальнейшего статического анализа программ.

5. Исходные файлы

Исходные файлы проекта: репозиторий [Электронный ресурс]. — GitHub. — Режим доступа: https://github.com/Vsevomes/ProgLang_lab1/tree/lab2 (дата обращения: 01.12.2025).